



# ME 604 - ROBOTICS

Start



# SUMMARY TILL PRESENTATION 2





# Ball Shooter Robot using a Flywheel Launcher

## Aim:

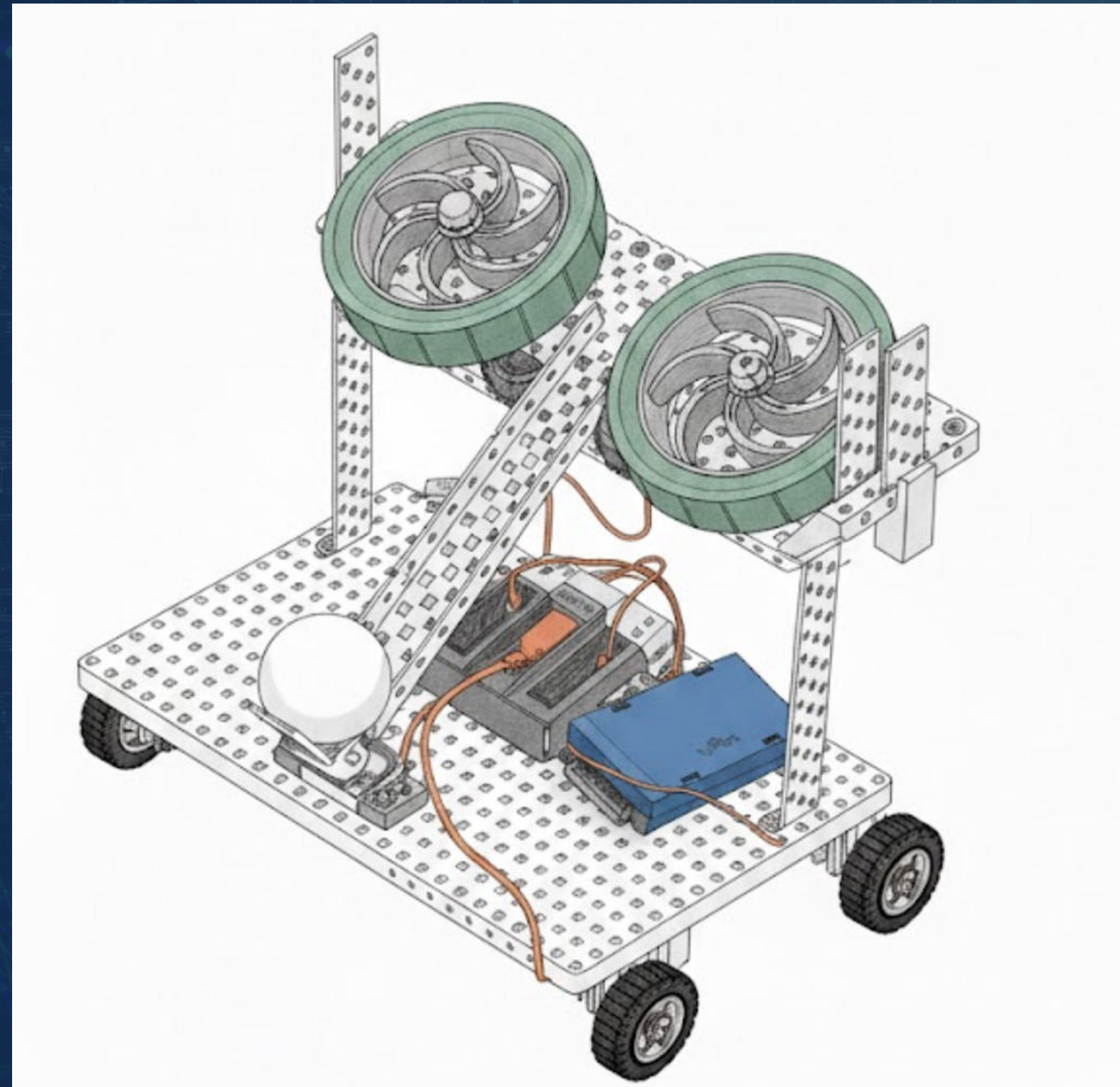
Build a compact two-axis (pan + tilt) aiming platform with a flywheel launcher capable of autonomously detecting a desk-height hoop and shooting a light ball (tennis-type) from 1.5–4.0 m.

## Objectives/Target Specification:

- Demonstrate  $\geq 80\%$  success rate (8/10) for shots into a desk-height hoop (inner dia  $\approx 12$  cm) from horizontal distances 1.5 - 4.0 m after calibration.
- Mean landing error  $\leq \pm 5$  cm from hoop center (measured over 10 shots at each distance).
- Actuation accuracies: pan  $\leq \pm 1^\circ$ , pitch  $\leq \pm 1^\circ$
- Flywheel launch speed repeatability  $\pm 0.1$  m/s (kind of  $< 5\%$  of launch speed).
- Perception-to-actuation latency  $\leq 3$  sec
- Simulation: MATLAB (Robotics Toolbox) model for aiming + projectile ballistics.
- Total hardware cost  $\leq ₹ 5,000$



# CONCEPT SKETCH

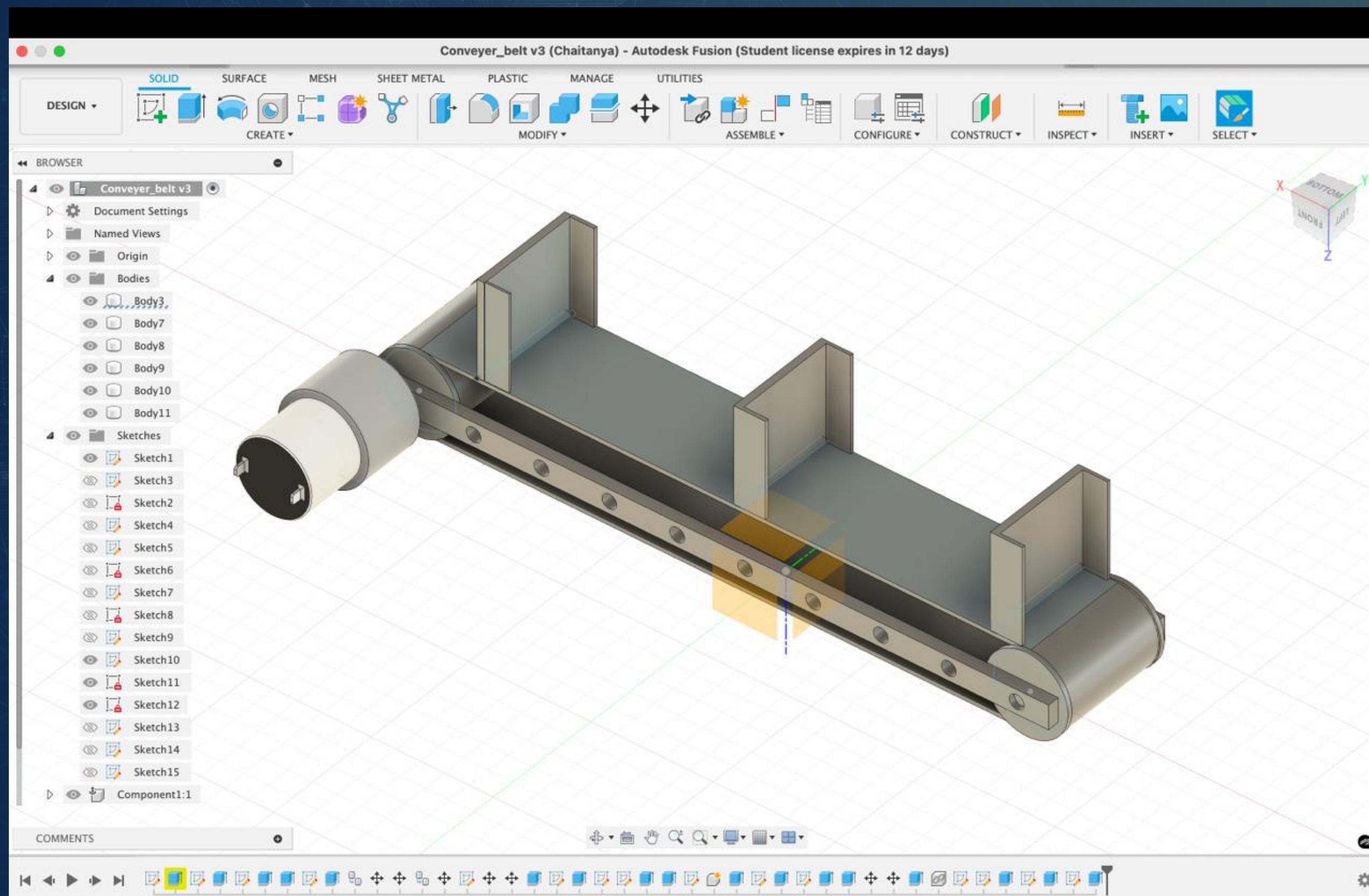




# CAD

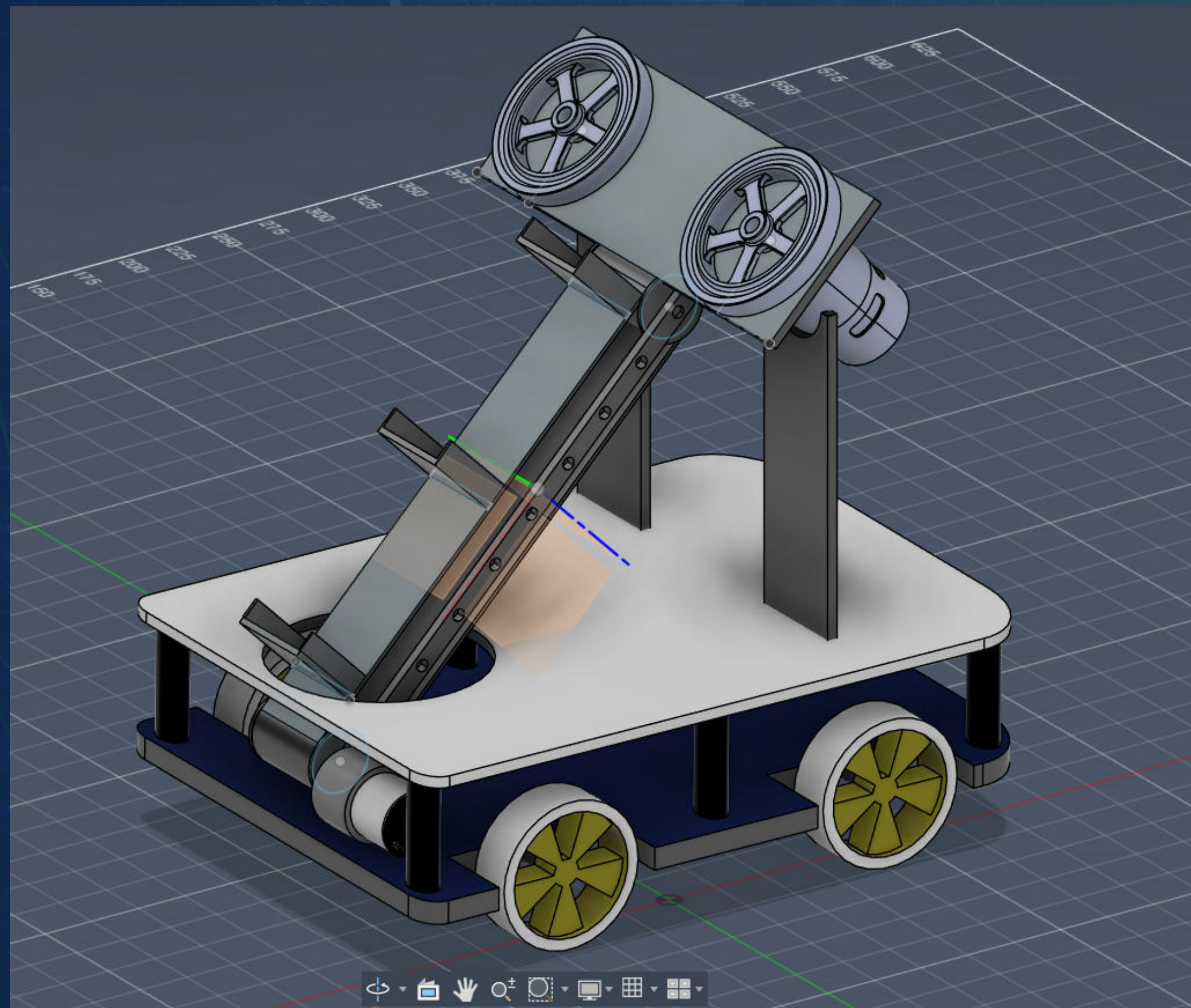


## THE BALL LOADING MECHANISM





# CAD





# PRESENTATION 3

## NEW WORK





# EXP SETUPS



Voltage and current required (CH2) for working of the robot

Testing ESP CAM max range for detecting the basket ball rectangle with good accuracy



# SHOOTER MECH



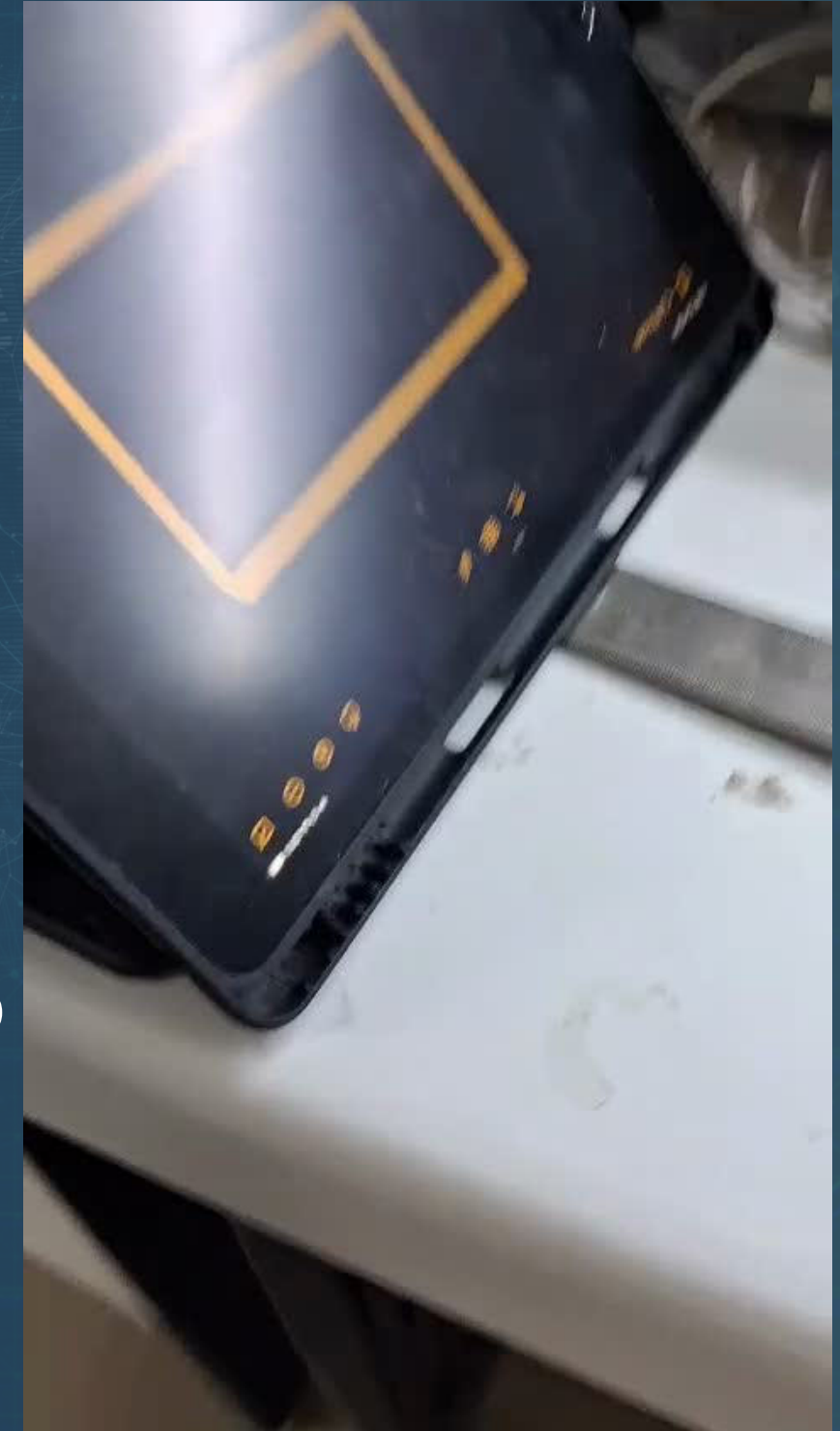


# PUSHING MECH AND DETECTION EXP



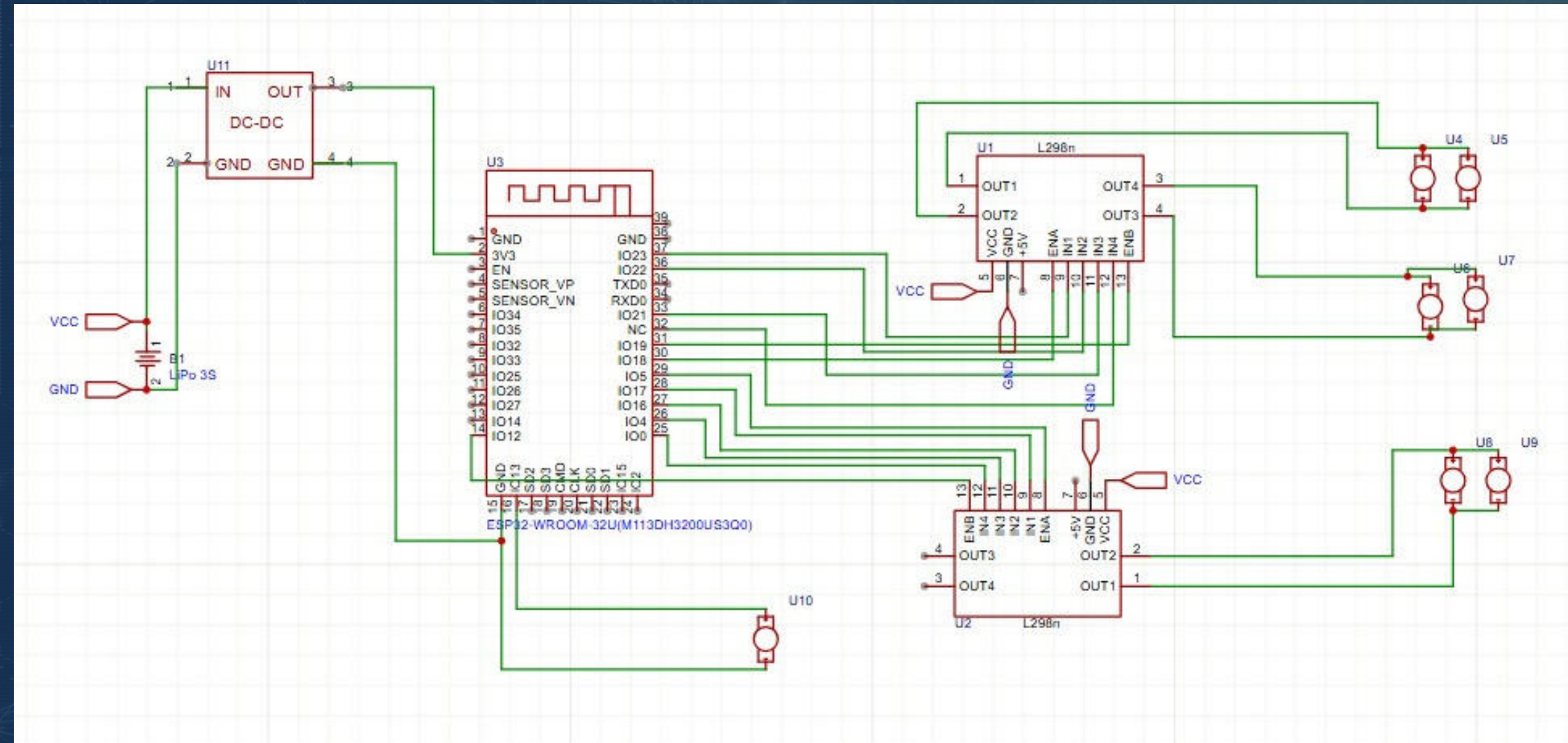
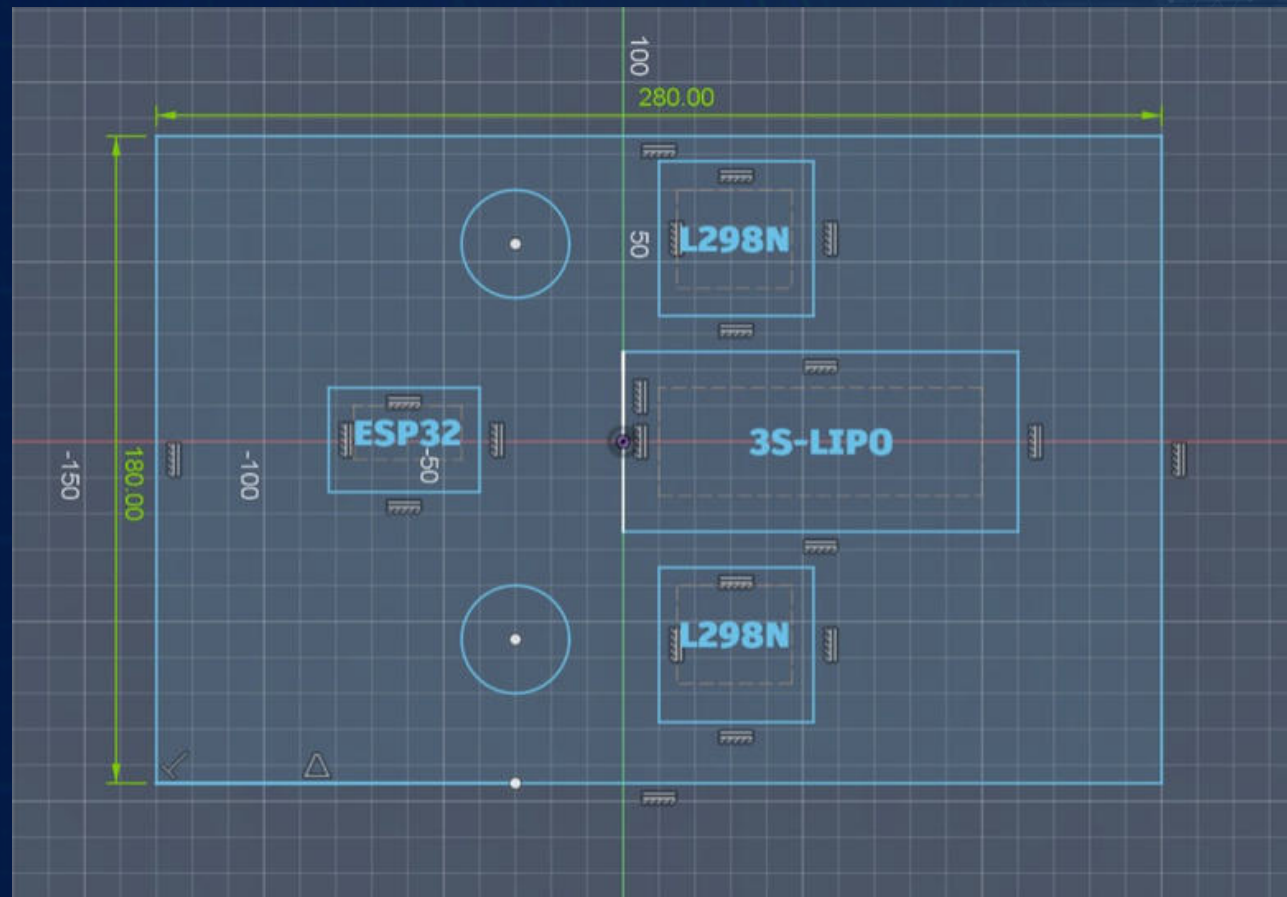
Linear actuator  
system for loading  
the ball

Detection of  
basketball hoop





# ELECTRONIC SETUP



SKU: 1137795

1. Operating Voltage(V): 12
2. Rated Speed (RPM): 1000
3. Rated Torque(kg-cm): 0.3
4. Stall Torque(kg-cm): 1.5
5. No Load Current (A): 0.06
6. Load Current (A): 0.3



# WORKDONE LISTED



- Base robot fabricated ✓
- Chassis + wheel system ready
- Stable platform achieved
- Compact layout implemented
  
- Flywheel shooter built (dual motor)
- Linear pusher loading system
- Shift from the belt to a linear mechanism
- Improved control & consistency
- Enables advanced shots (curved shots)





# PROCUREMENT LIST UPDATED



| Name of Component              | Quantity | Procurement Status |
|--------------------------------|----------|--------------------|
| 12V DC 200 RPM Motor & Wheels  | 2        | Procured           |
| L298N Motor Driver             | 2        | Procured           |
| ESP 32                         | 1        | Procured           |
| BO 105 motor (Ball feeding)    | 1        | Procured           |
| wheel base chasis              | 1        | Built              |
| ESP 32 Camera module xMp       | 1        | Procured           |
| 12v DC 5K RPM Motor (Shooting) | 2        | Procured           |





# PROCUREMENT STATUS

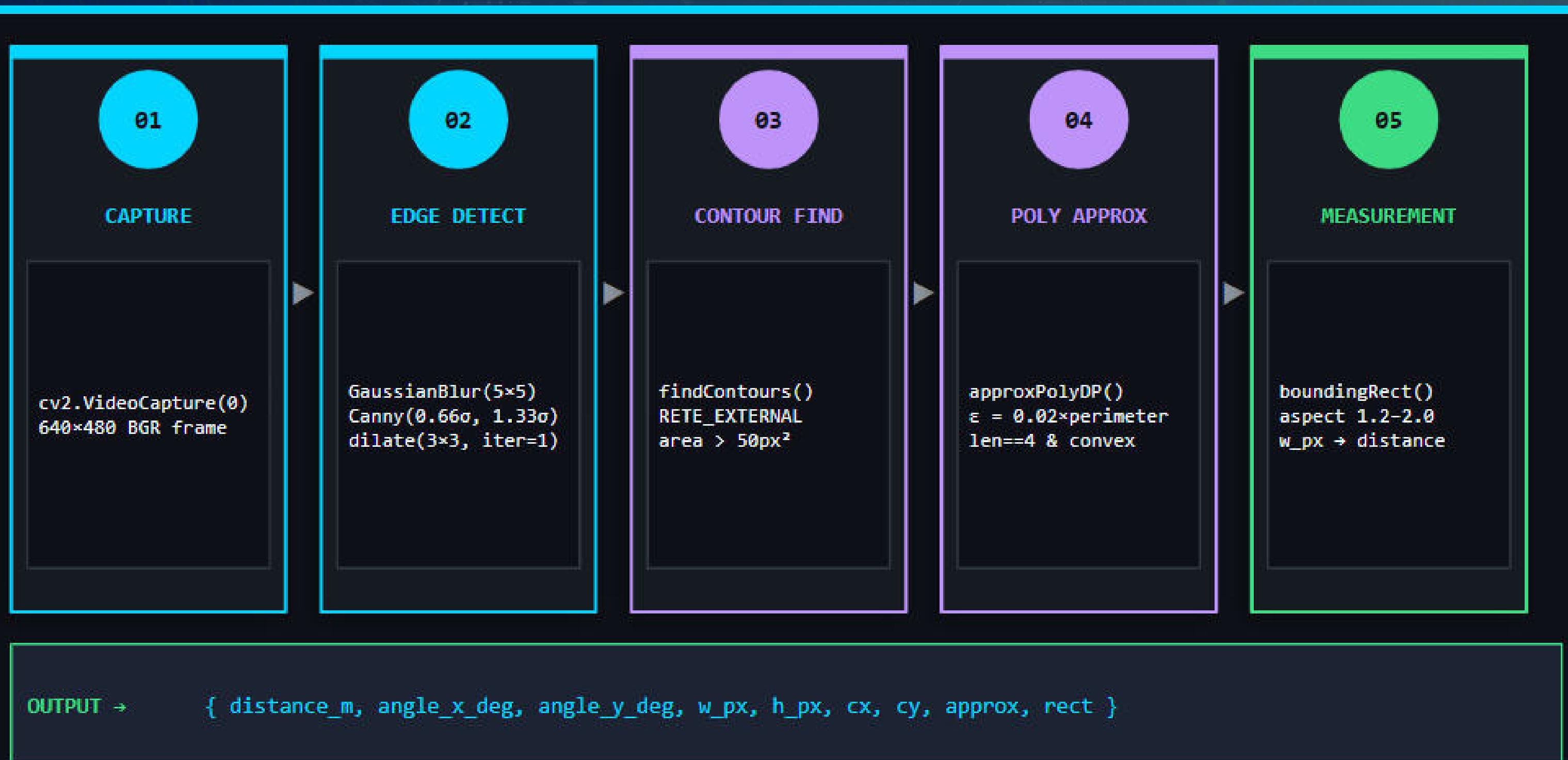


| Name of Component                               | Quantity | Procurement Status |
|-------------------------------------------------|----------|--------------------|
| 3S Lithium Ion Battery                          | 1        | Procured           |
| Required Buck converters, Capacitors, Resistors | 1        | Procured           |
| 10cm diameter Flywheel with rubber grip         | 2        | Procured           |





# DISTANCE ESTIMATION USING CAMERA





# DISTANCE ESTIMATION USING CAMERA



The image shows a VS Code editor with a Python script named `hoop_detector.py` and a terminal window. The script calculates distance and angles based on camera parameters and object dimensions.

```
37 # focal_px = (frame_width / 2) / tan(FOV_h / 2)
38 FOCAL_LENGTH_PX = 575.0 # calibrated: 69px at 1m with 12cm rect
39
40
41 #
42 def calculate_distance(apparent_diameter_px: float) -> float:
43     """
44     Distance via pinhole model:
45     D = (real_diameter * focal_length) / apparent_diameter_px
46     """
47     if apparent_diameter_px <= 0:
48         return float('inf')
49     return (RECT_REAL_WIDTH_M * FOCAL_LENGTH_PX) / apparent_diameter_px
50
51
52 def calculate_angles(cx_px: float, cy_px: float) -> tuple[float, float]:
53
54     dx = cx_px - FRAME_WIDTH / 2.0
55     dy = FRAME_HEIGHT / 2.0 - cy_px # invert Y so up = positive
56
```

The terminal window shows the output of the script, displaying distance and angle values for multiple frames:

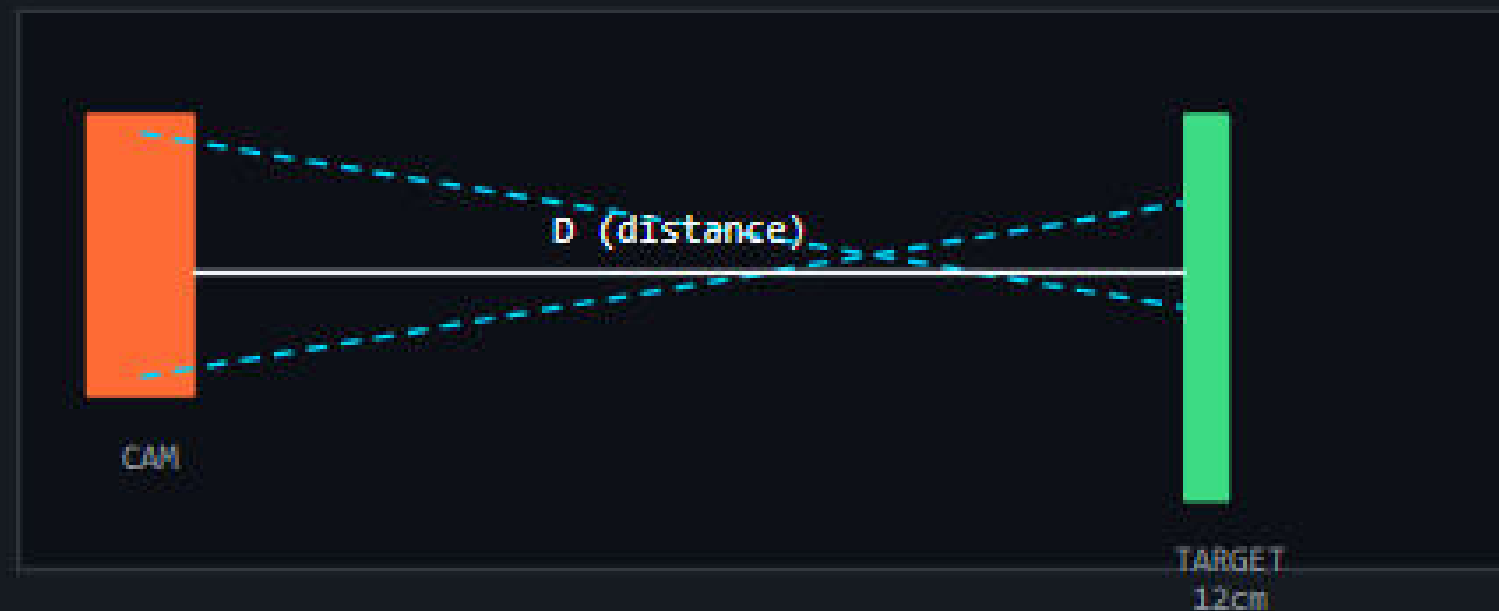
| PROBLEMS | OUTPUT     | DEBUG CONSOLE | TERMINAL | PORTS      |
|----------|------------|---------------|----------|------------|
| [HOOP]   | dist=0.21m | ax=-4.6°      | ay=+1.6° | w_px=327.0 |
| [HOOP]   | dist=0.21m | ax=-4.6°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.6°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.6°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.6° | w_px=328.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.6° | w_px=327.0 |
| [HOOP]   | dist=0.21m | ax=-4.4°      | ay=+1.8° | w_px=327.0 |
| [HOOP]   | dist=0.21m | ax=-4.4°      | ay=+1.8° | w_px=327.0 |
| [HOOP]   | dist=0.21m | ax=-4.4°      | ay=+1.8° | w_px=326.0 |
| [HOOP]   | dist=0.21m | ax=-4.4°      | ay=+1.8° | w_px=326.0 |
| [HOOP]   | dist=0.21m | ax=-4.4°      | ay=+1.8° | w_px=326.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.8° | w_px=326.0 |
| [HOOP]   | dist=0.21m | ax=-4.5°      | ay=+1.8° | w_px=326.0 |

The `Hoop Detector` window shows a video feed of a hoop with a green bounding box and a red dot indicating the center. The output text is:

```
Distance : 0.21 m
Angle X : -4.5 deg
Angle Y : +1.8 deg
Rect W/H : 326 / 209 px
```

## CAMERA CALIBRATION & DISTANCE MODEL

### PINHOLE CAMERA MODEL



Focal Length (auto)

$$f = (W/2) / \tan(FOV_h / 2)$$

Focal Length (calibrated)

$$\begin{aligned} f &= (w_{px} \times D) / W_{real} \\ &= (69.0 \times 1.0) / 0.12 \\ &= 575.0 \text{ px} \end{aligned}$$

Distance

$$D = (W_{real} \times f) / w_{px}$$

### ANGLE CALCULATION

Pixel offset (horizontal)

$$dx = cx - \text{FRAME\_WIDTH} / 2$$

Pixel offset (vertical)

$$dy = \text{FRAME\_HEIGHT} / 2 - cy$$

Horizontal angle

$$\text{angle\_x} = \text{atan2}(dx, f)$$

Vertical angle

$$\text{angle\_y} = \text{atan2}(dy, f)$$

Sign convention

+angle\_x → hoop is RIGHT  
+angle\_y → hoop is ABOVE

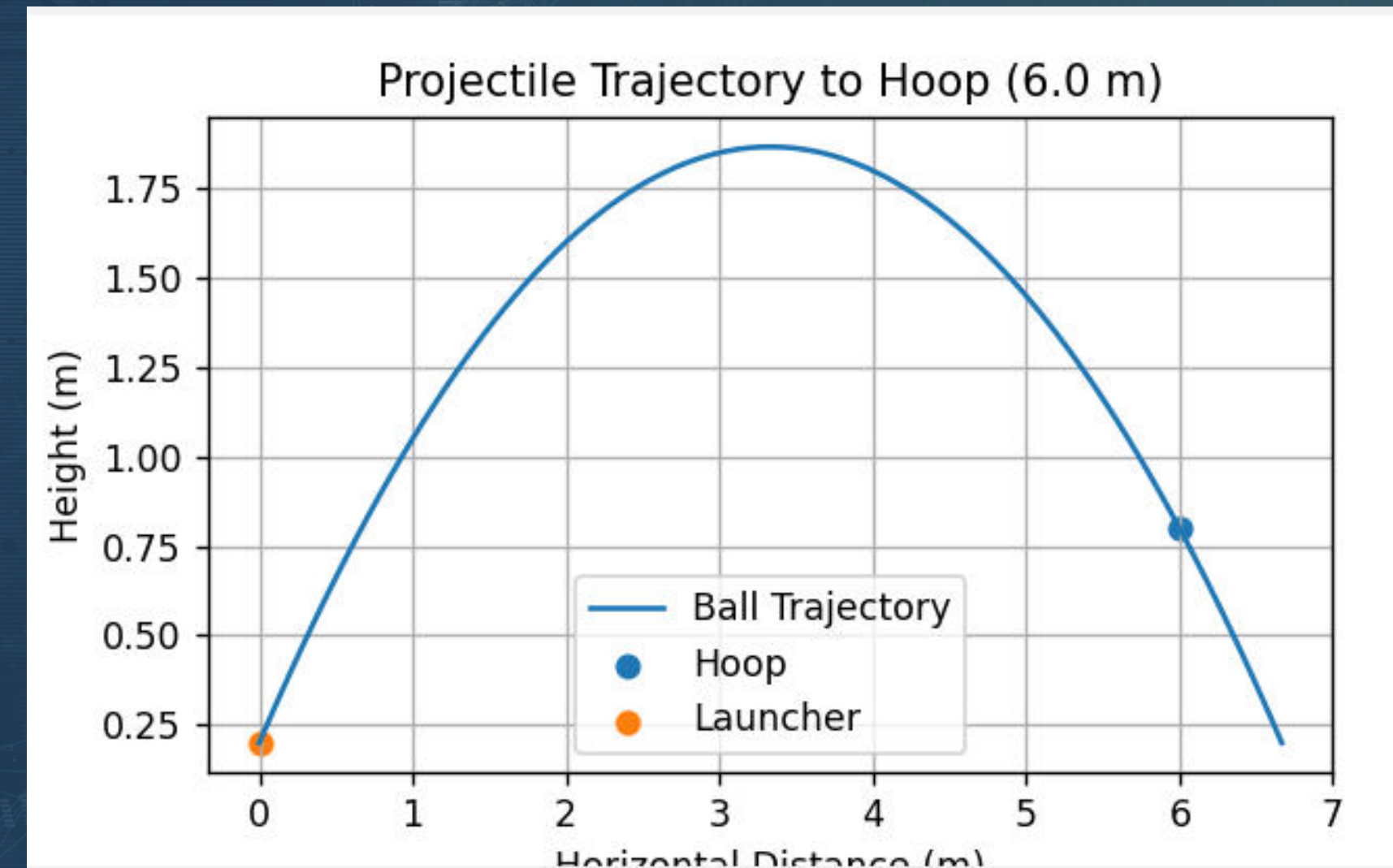


# RESULTS OF TRAJECTORY ESTIMATION AND RPM CALCULATION



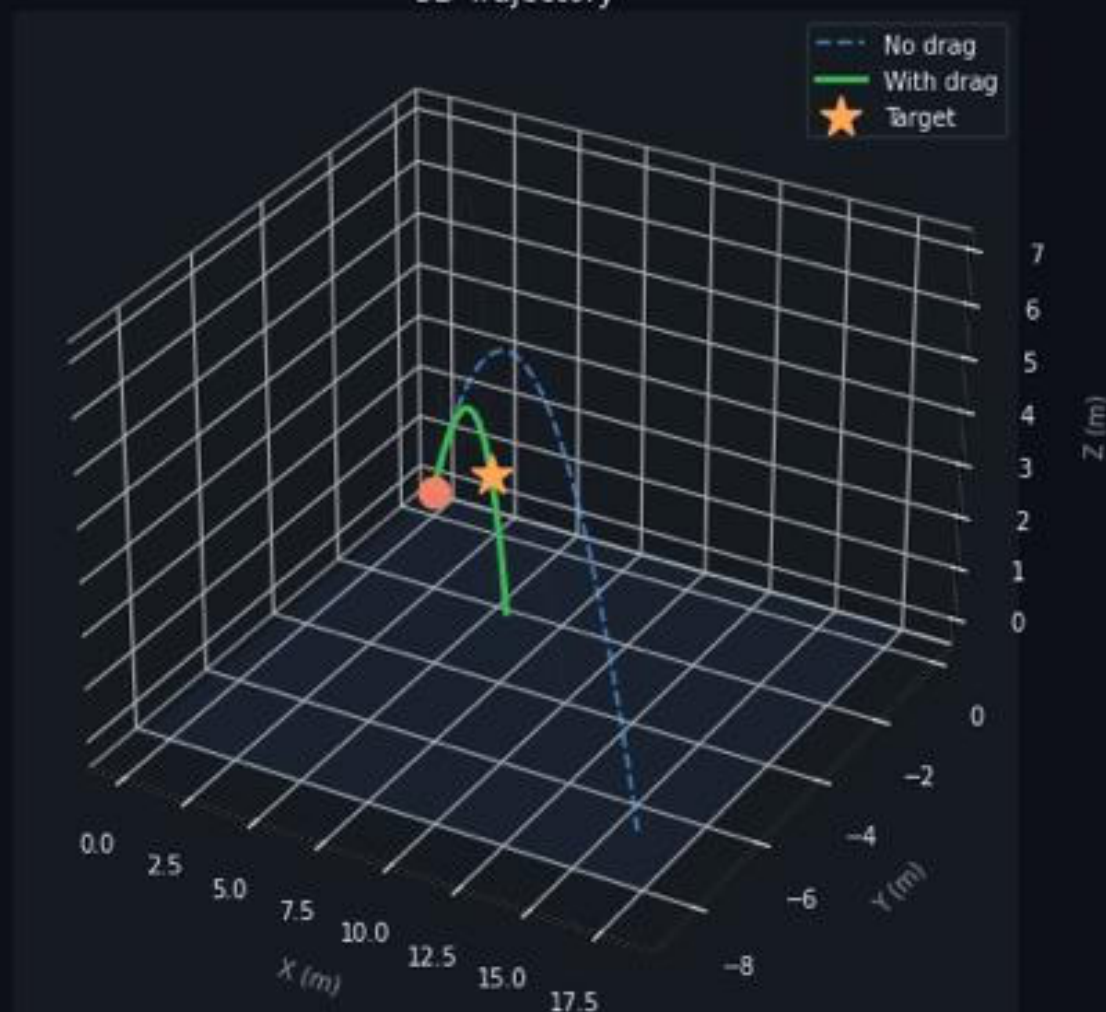
Launch angle :  $45.0^\circ$   
Launch height: 0.2 m  
Hoop height : 0.8 m  
Flywheel  $\phi$  : 10 cm

| Dist (m) | $v_0$ (m/s) | RPM  | Flight(s) | Apex(m) | Valid |
|----------|-------------|------|-----------|---------|-------|
| 2.0      | 5.29        | 1189 | 0.534     | 0.91    | ✓     |
| 2.5      | 5.68        | 1276 | 0.622     | 1.02    | ✓     |
| 3.0      | 6.07        | 1362 | 0.699     | 1.14    | ✓     |
| 3.5      | 6.44        | 1446 | 0.769     | 1.26    | ✓     |
| 4.0      | 6.79        | 1526 | 0.833     | 1.38    | ✓     |
| 5.0      | 7.47        | 1677 | 0.947     | 1.62    | ✓     |
| 6.0      | 8.09        | 1817 | 1.049     | 1.87    | ✓     |

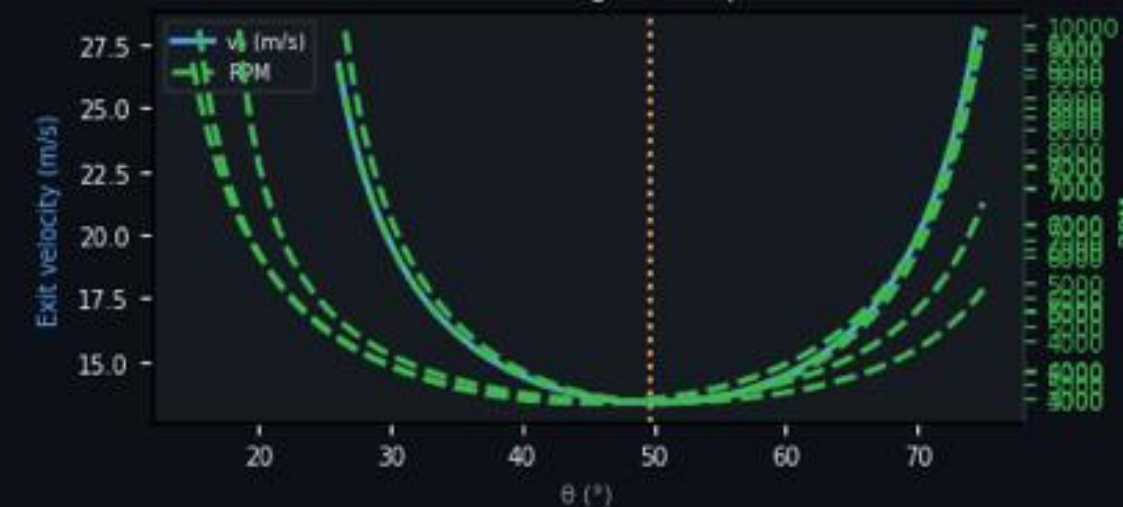


# Ping Pong Ball Launcher — Physics Simulator

3D Trajectory



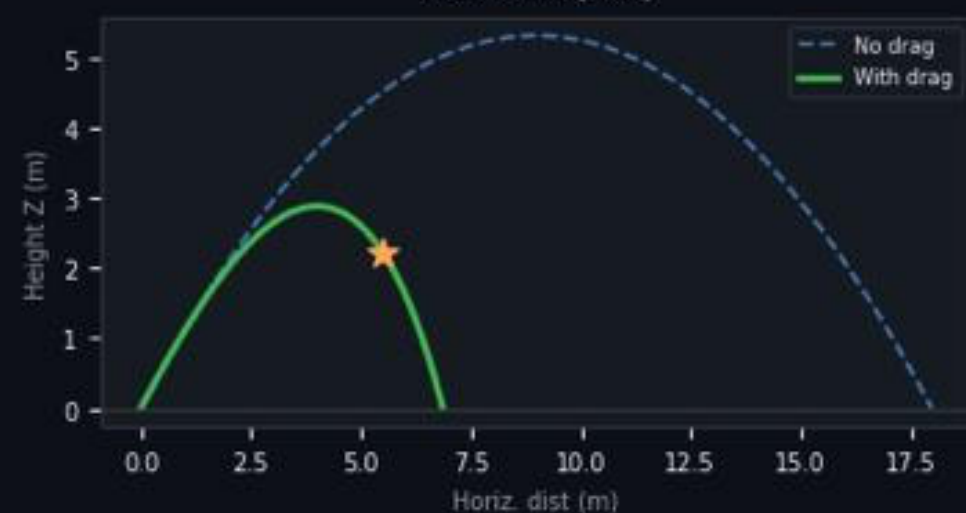
Launch Angle Sweep



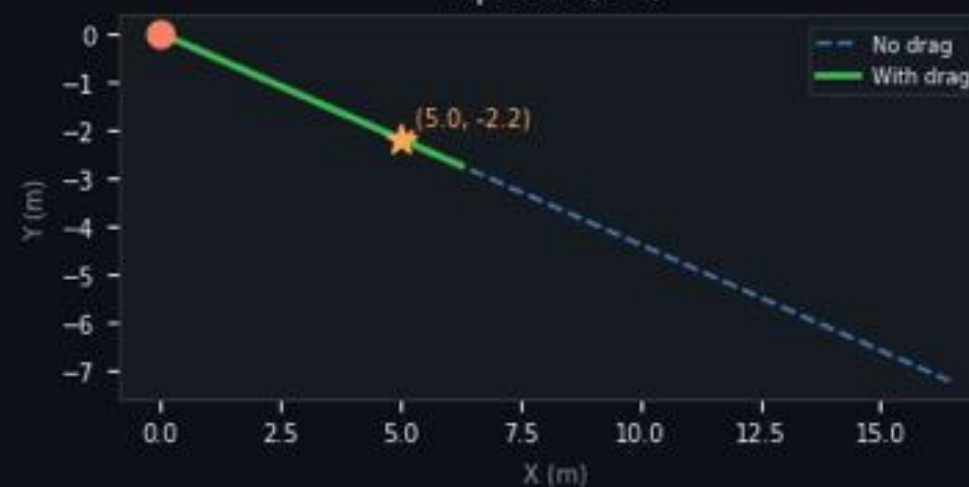
Motor RPM



Side View (X-Z)



Top View (X-Y)



Motor Commands [drag-solved]

|                   |                       |
|-------------------|-----------------------|
| TARGET:           | (5.03, -2.22, 2.22) m |
| Distance:         | 5.50 m                |
| $\theta$ (pitch): | 49.8°                 |
| $\phi$ (yaw):     | -23.8°                |
| $v_0$ :           | 13.37 m/s             |
| RPM:              | 4729                  |
| $\omega$ :        | 495.2 rad/s           |
| Solver err:       | 0.0006 m              |
| Status:           | ON TARGET             |

Target X (m)

Target Y (m)

Target Z (m)

Reset

1

HOOP DETECTION

5 consecutive frames confirmed | `detect_hoop(frame) → info dict`

✓ `distance_m` in `[0.5, 6.0]` m

2

PHYSICS SOLVE

`solve_shot(distance_m) → v0, RPM, flight_time, apex_height`

✓ `RPM ≤ MAX_FLYWHEEL_RPM (6000)`

3

BOT ALIGNMENT

PID loop @ 20Hz | differential drive | `angle_x → 0°`

✓ `abs(angle_x) < 2.0°`

4

RE-CONFIRM AIM

Re-detect after movement | recalculate RPM with updated distance

✓ `hoop still visible`

5

SHOOT SEQUENCE

`spinup(RPM) → load_ball() → fire() → spindown()`

✓ `sequence complete`



# FUTURE WORK (TODO)



- 3D print clamp for ESP and ESP-CAM to mount on the robot.
- Modify the shaft of 5k RPM motors to extend the shaft length and to be able to fit the wheel.
- Organize the wiring and make it look clean and tidy.
- Make the small scaled basket ball hoop according to the size of the ping pong ball.
- Compare the range and velocity difference between 1k RPM and 5k RPM, physically.
- Mitigate the vibrations in the shooter mechanism – the prominent reason being the misalignment of the wheel and the motor shaft.
- More feedback from prof.





# WHAT TO DO NEXT



**Fabrication**



**Assembly**



**Designing of electrical  
circuit /connections**



**Testing and calibration**



**Simulation of bot**





# THANK YOU!

